

Homework 4: Transformations Mapping Pipeline

4.1 - Scaling and Rotating Patches

4.1.1 - Concepts

Question: If I have a feature located at (x, y) with rotation θ and scale s , what is the transformation matrix H that simultaneously moves the feature to the origin, un-rotates it, and un-scales it (so that the feature becomes 1 pixel wide)?

In order to “undo” each of these transformations, we can first find the matrix that undoes each individual transformation, and then combine them into a single matrix that does all three at once. Below are the three matrices to undo transformation, rotation, and scaling individually:

$$H_{\text{scale}} = \begin{bmatrix} 1/s & 0 & 0 \\ 0 & 1/s & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad H_{\text{rotate}} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad H_{\text{transform}} = \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix}$$

To combine these transformations, we can simply multiply the three matrices together, like this:

$$H = H_{\text{scale}} H_{\text{rotate}} H_{\text{transform}}$$

Performing this matrix multiplication, we get the following matrix H :

$$H = \begin{bmatrix} \frac{\cos \theta}{s} & \frac{\sin \theta}{s} & \frac{-\cos \theta x_f - \sin \theta y_f}{s} \\ \frac{-\sin \theta}{s} & \frac{\cos \theta}{s} & \frac{\sin \theta x_f - \cos \theta y_f}{s} \\ 0 & 0 & 1 \end{bmatrix}$$

4.1.2 - Implementation

Below is my implementation of the transformation matrix, following the structure from 4.1.1:

```
sin_t = np.sin(feature_orientation)
cos_t = np.cos(feature_orientation)
transformation_matrix = [
    [cos_t / feature_radius, sin_t / feature_radius, ((-cos_t * feature_center_x) - (sin_t * feature_center_y)) / feature_radius],
    [-sin_t / feature_radius, cos_t / feature_radius, ((sin_t * feature_center_x) - (cos_t * feature_center_y)) / feature_radius],
    [0, 0, 1],
]
```

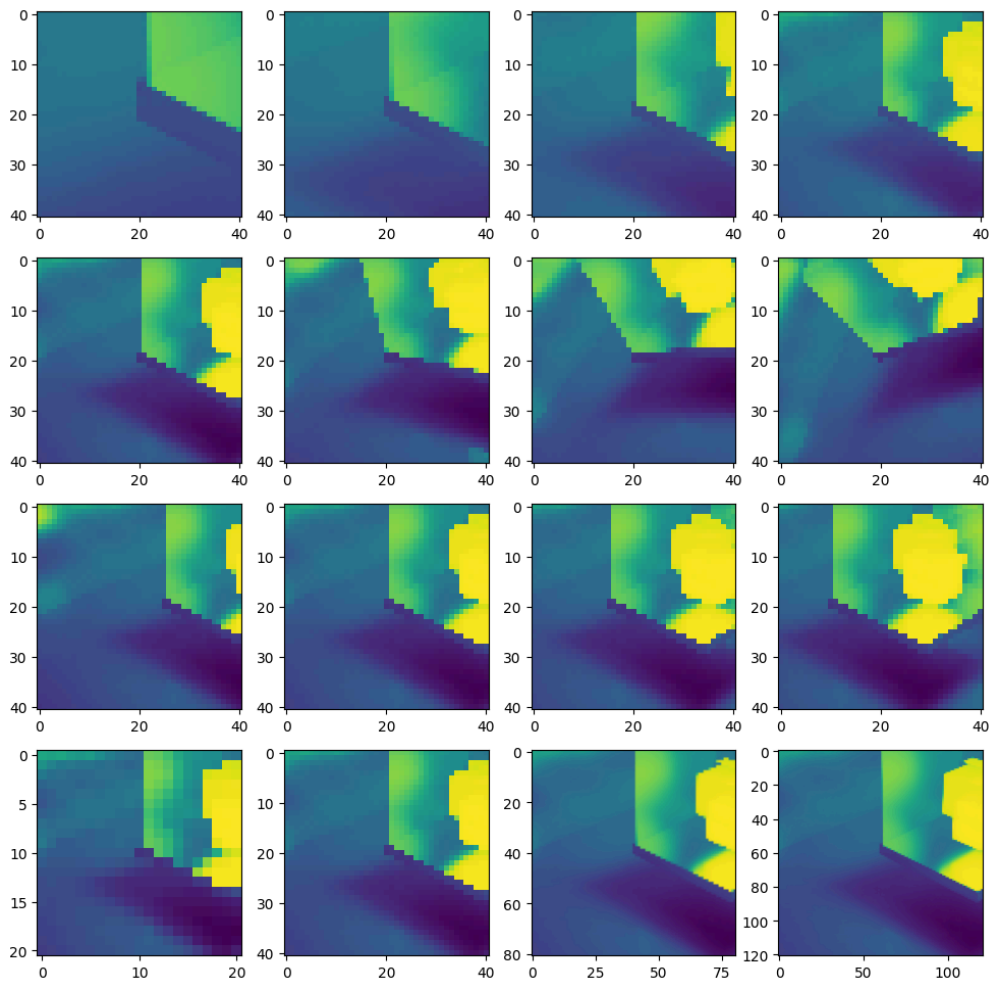
And here is my implementation for the inverse transformation to get the resulting x and y positions for the transformed pixels:

```
for ii in range(len(xi)):
    for jj in range(len(yi)):

        # Apply the inverse transformation matrix to the point
        new_x, new_y, _ = H_inv @ [xi[ii], yi[jj], 1.0]

        if new_x is None or new_y is None:
            raise NotImplementedError(
                "You must define the new coordinates")
        # Don't forget to rehomogenize!
        patch[jj, ii] = image_fn(new_x, new_y)[0, 0]
```

Lastly, here is a figure of the resulting patches with base center position (500, 640):



4.2 - Computing Homographies from Matches

4.2.1 - Perfect Matches

Here is my implementation of a method that uses singular value distribution to take a provided list of matches and reverse-engineer the matching homography matrix:

```
def solve_homography(matches: list[list[int]]):
    """Compute the homography matrix that maps to the provided matches
    | Each match: [x, y, x', y']"""

    N = len(matches)
    A = np.zeros((2*N, 9))

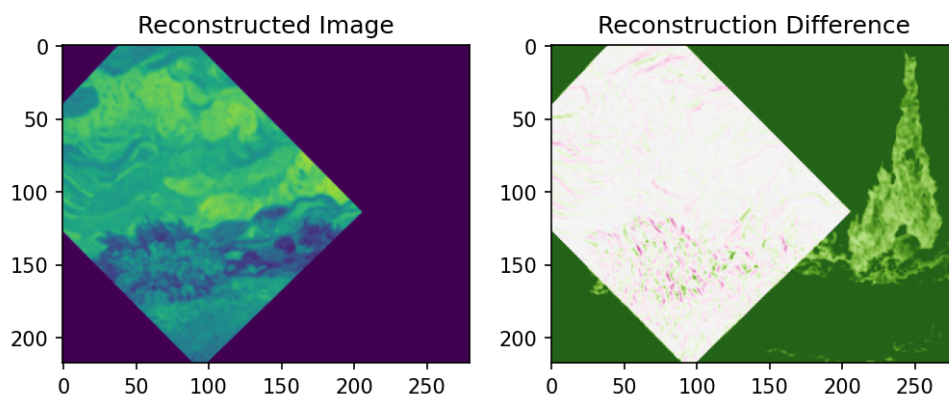
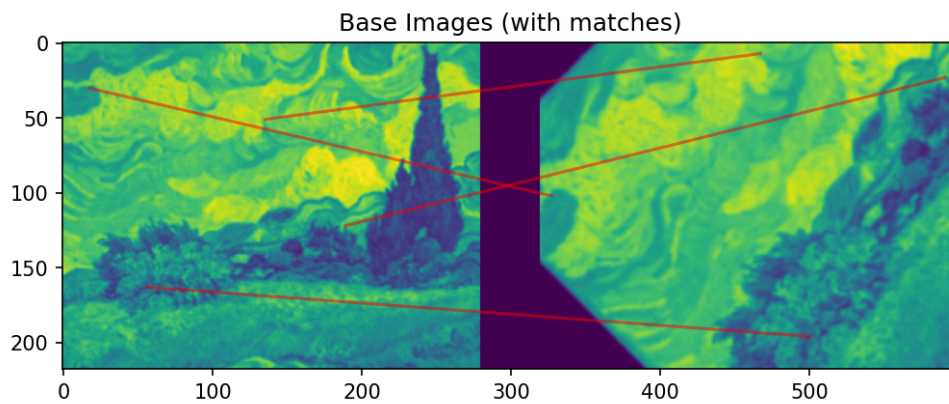
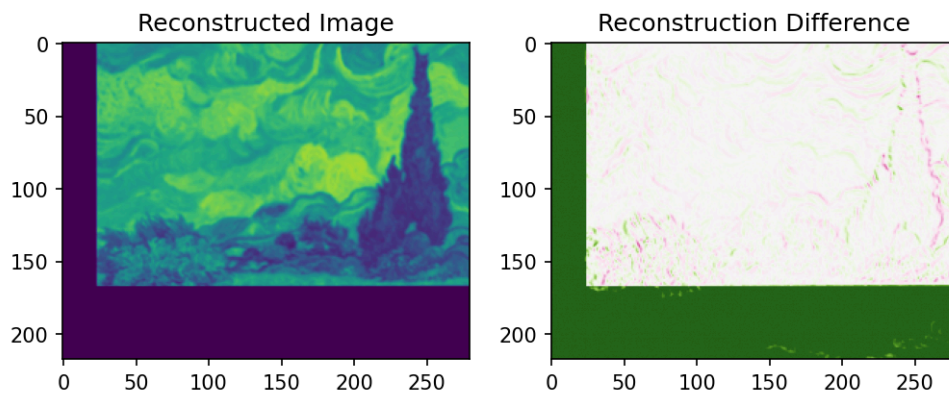
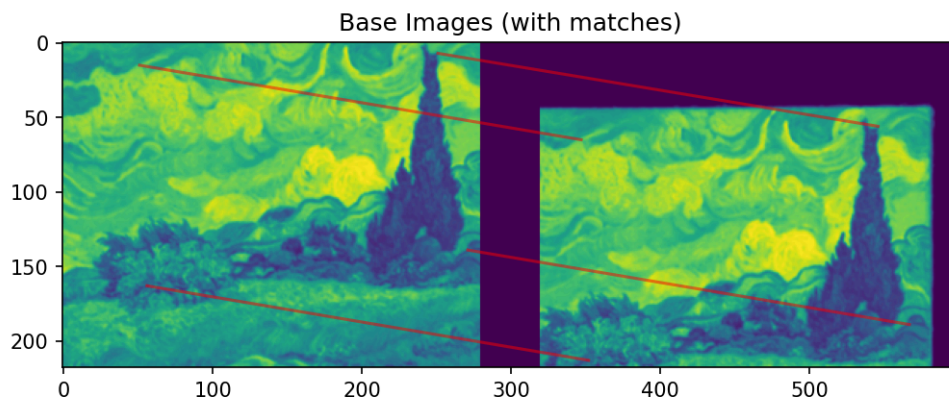
    # Build full matrix form (based on lecture slides)
    for i, (x, y, xp, yp) in enumerate(matches):
        A[2*i] = [x, y, 1, 0, 0, 0, -xp*x, -xp*y, -xp]
        A[2*i + 1] = [0, 0, 0, x, y, 1, -yp*x, -yp*y, -yp]

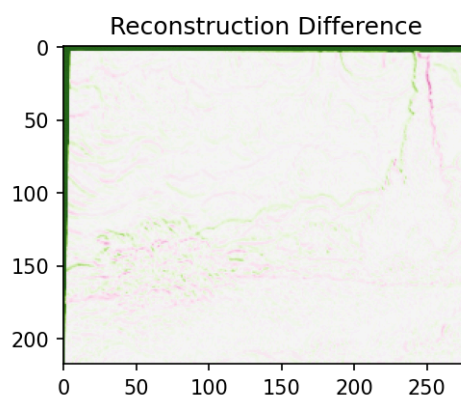
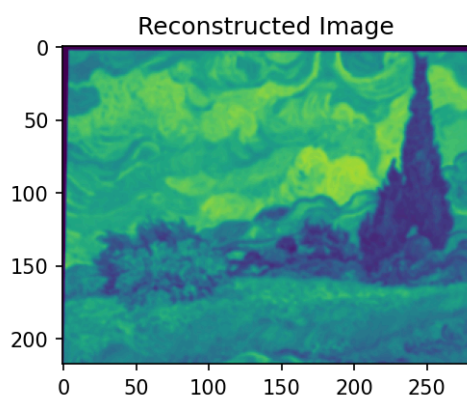
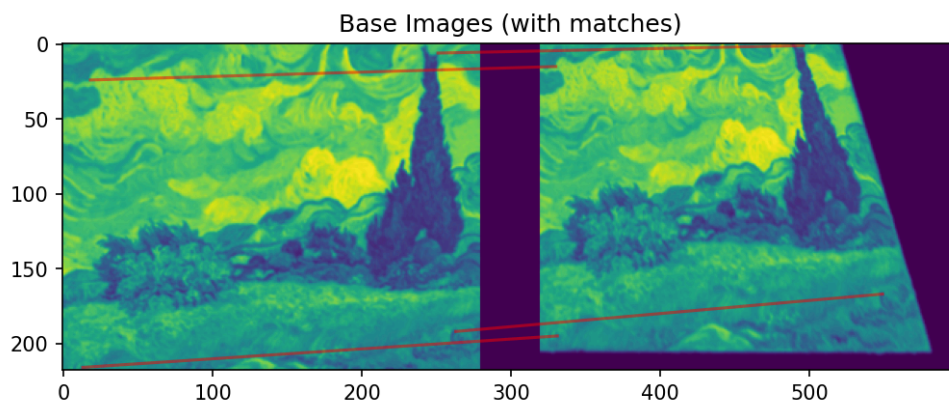
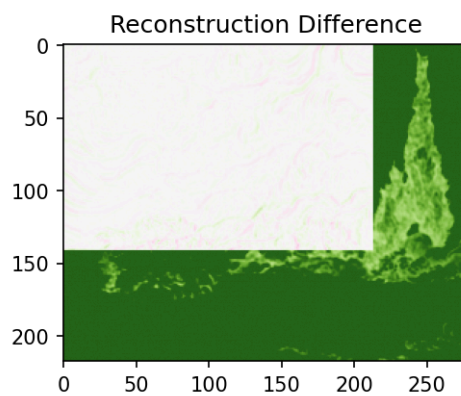
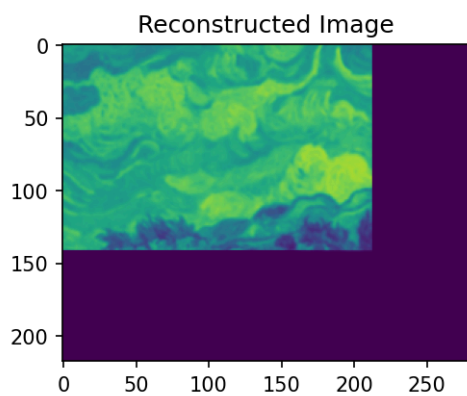
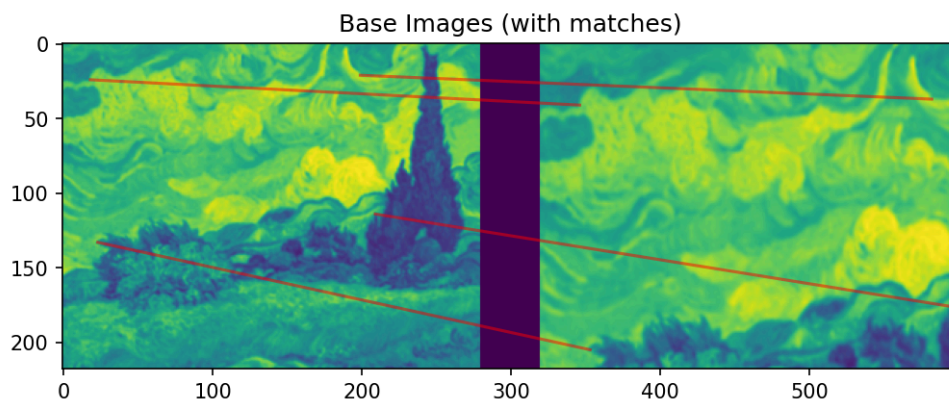
    # Solve Ah = 0 using singular value distribution
    # (The solution H is the last row of the SVD result)
    _, _, solution = np.linalg.svd(A)
    H_flat = solution[-1]

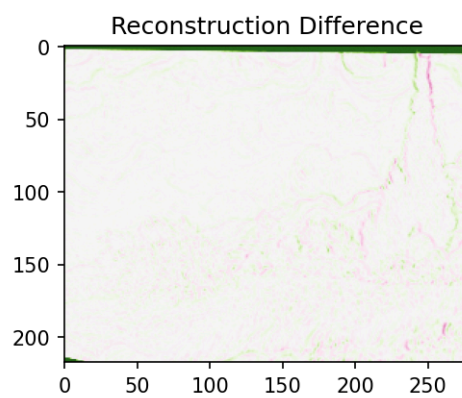
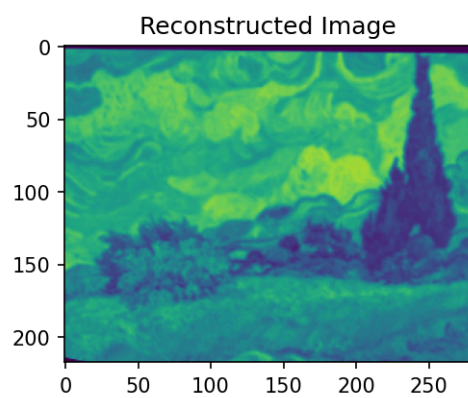
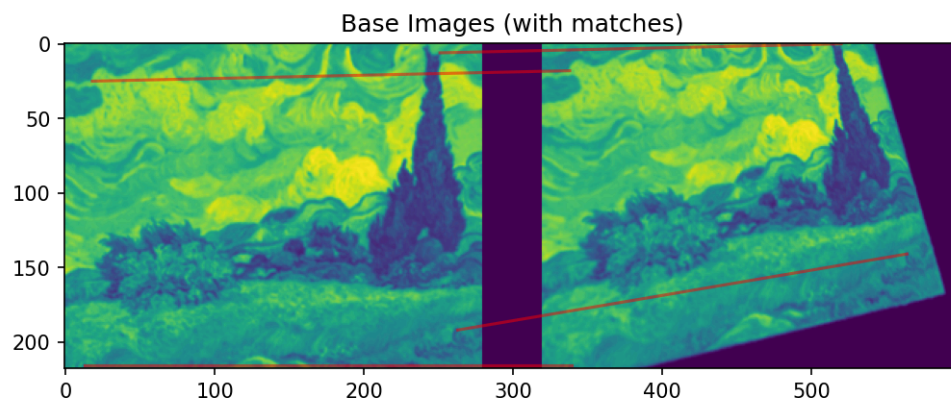
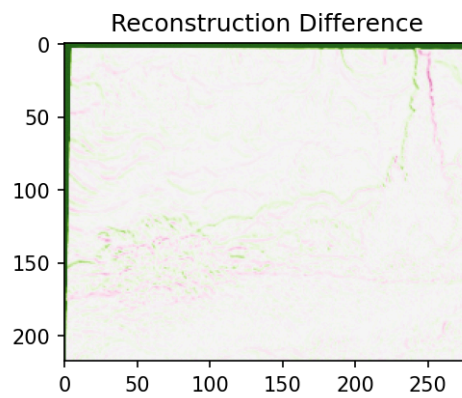
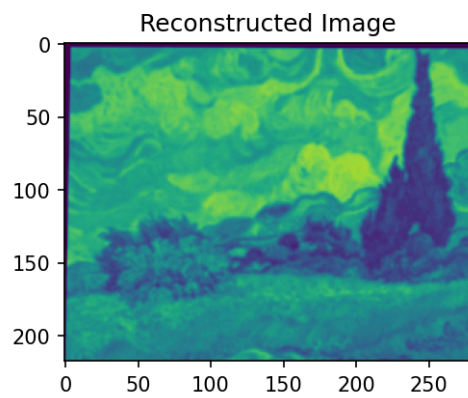
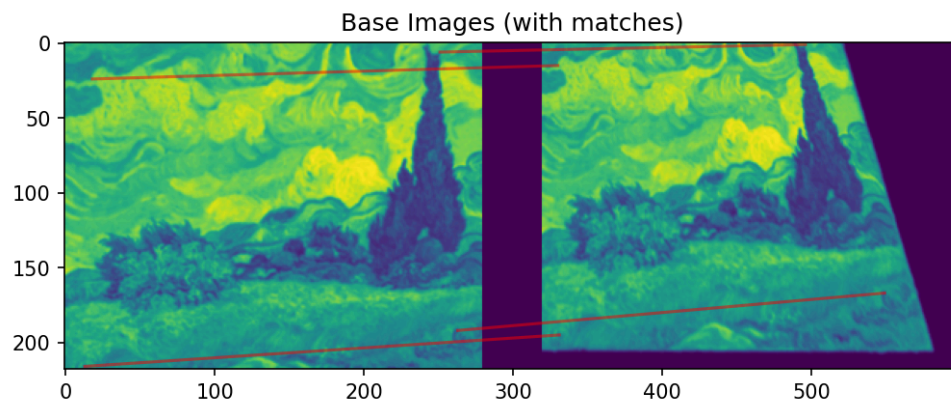
    # Reshape the 1x9 H result into a 3x3 homography matrix
    H = np.array([
        [H_flat[0], H_flat[1], H_flat[2]],
        [H_flat[3], H_flat[4], H_flat[5]],
        [H_flat[6], H_flat[7], H_flat[8]],
    ])

    # Normalize to match the provided example (H[2, 2] = 1)
    H /= H[2, 2]
    return H
```

Below are examples from six different homographies of the same base image. The above function was used on hand-picked match points for each image (marked by the red lines), and gave the displayed reconstructed result. The reconstruction differences show that the approximations are quite close, with only minimal error in each image.

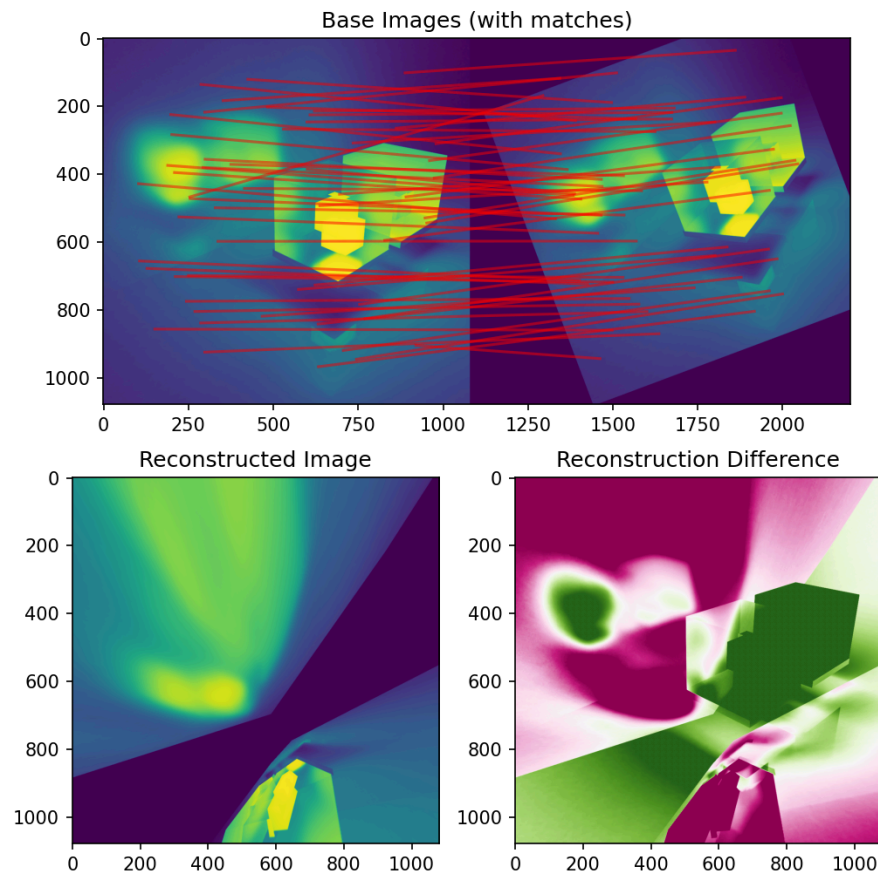






4.2.2 - Noisy Matches

Here is the result of solving the homography for the noisy match image using the simple method:



Below is my implementation of the more effective RANSAC procedure for computing a homography that is robust to outliers (logic based on diagrams/examples in slides):

```
def get_inliers(H, matches, threshold_px=5):
    """Find the matches that are inliers based on H (residuals within threshold)"""
    matches = np.asarray(matches, dtype=np.float64)
    source_points = matches[:, :2] # (x, y)
    transformed_points = matches[:, 2:4] # (x', y')

    # Convert to homogenous coords (x, y, 1)
    src_h = np.hstack([source_points, np.ones((source_points.shape[0], 1))])

    # Calculate predicted points with H
    proj = (H @ src_h.T).T
    proj = proj / proj[:, [2]] # divide by w to normalize
    predicted_xy = proj[:, :2]

    # Find the residuals (difference between predicted and actual points)
    residuals = np.linalg.norm(predicted_xy - transformed_points, axis=1)

    # Points with low residuals are inliers!
    inlier_mask = residuals < threshold_px
    return matches[inlier_mask], residuals
```

```
def solve_homography_ransac(matches, rounds=100, sigma=5, s=4):
    best_inliers = []
    best_H = None

    for _ in range(rounds):
        # Randomly pick s points
        sample = random.sample(list(matches), s)

        # Calculate a homography using that sample
        H = solve_homography(sample)

        # Count the inliers
        inliers, _ = get_inliers(H, matches, threshold_px=sigma)

        # Keep track of the model that gets the most inliers
        if len(inliers) > len(best_inliers):
            best_inliers = inliers
            best_H = H

    # Use the inliers to get the final homography
    H_final = solve_homography(best_inliers)

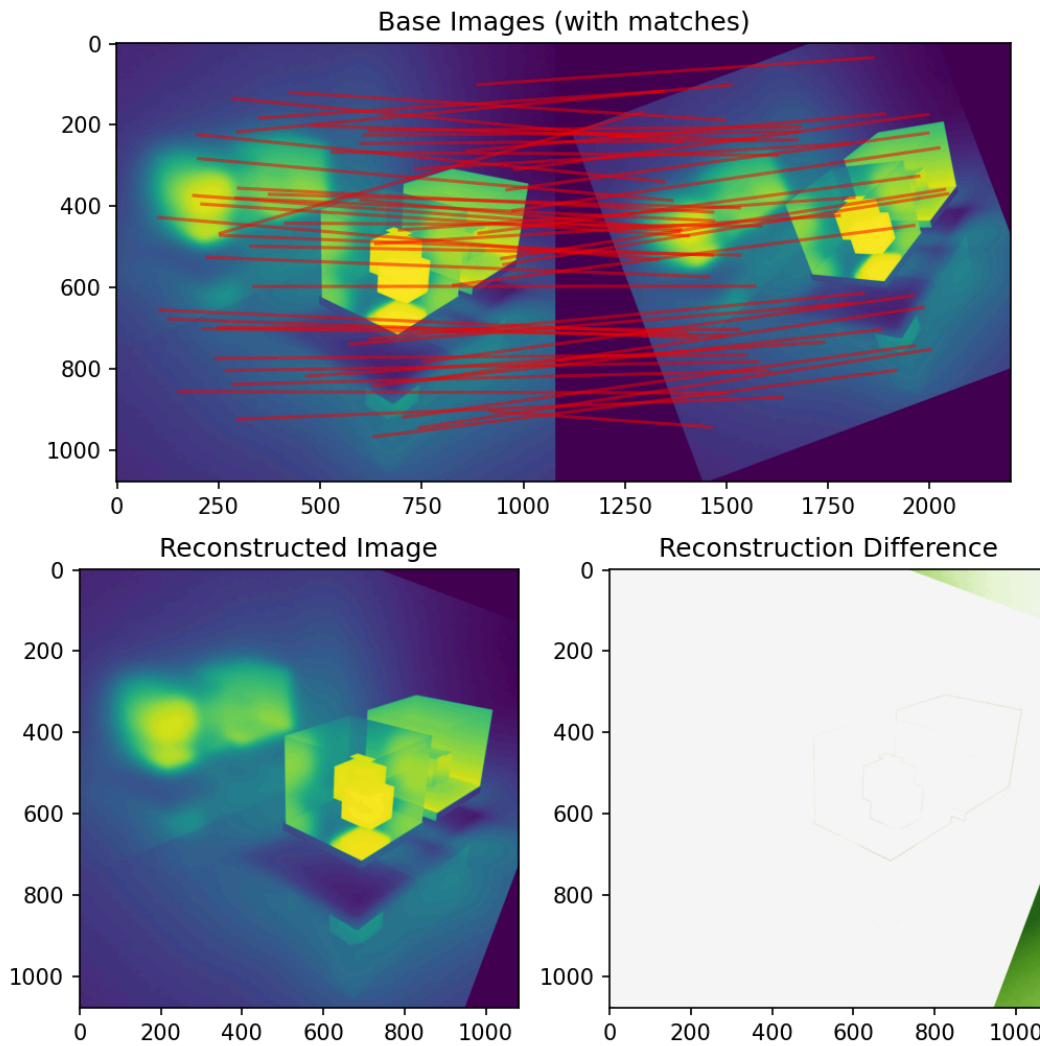
    # Normalize
    H_final /= H_final[2, 2]

    return H_final
```


Using the above RANSAC method, I calculated the following homography matrix:

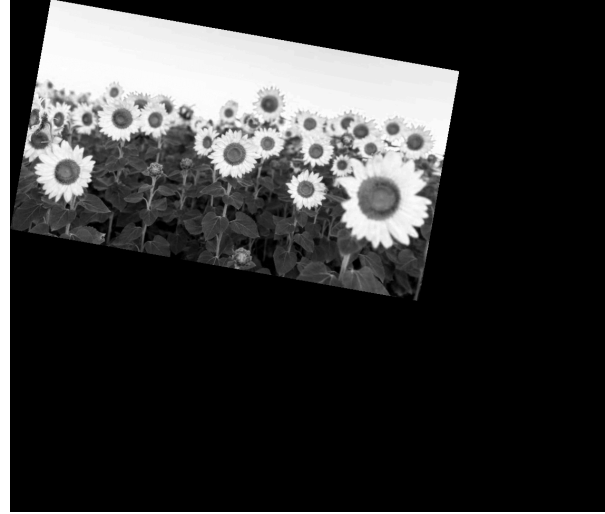
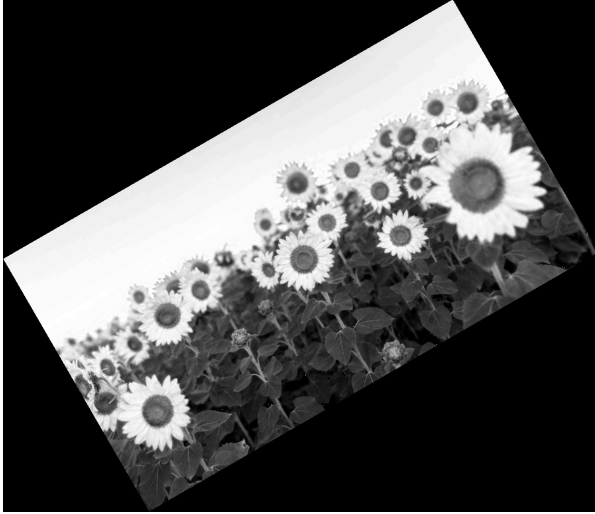
$$H_{\text{RANSAC}} = \begin{bmatrix} 0.8 & 0.3 & -2.63 \times 10^{-11} \\ -0.3 & 0.8 & 2.20 \times 10^2 \\ 4.77 \times 10^{-17} & 2.39 \times 10^{-17} & 1.0 \end{bmatrix}$$

And here is the result of applying that homography matrix to reconstruct the following image:



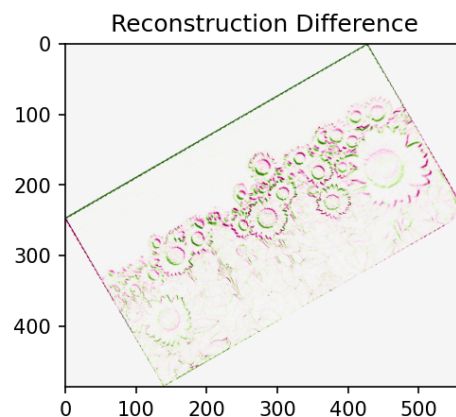
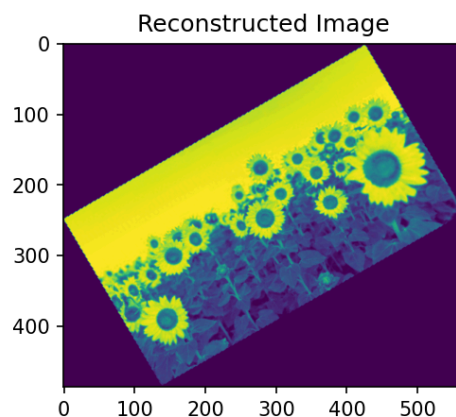
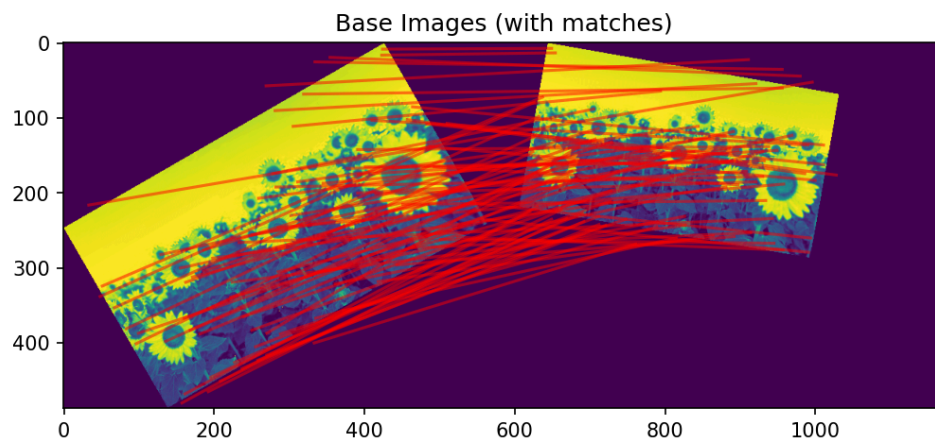
4.3 - Feature Matching Pipeline

I used the multi-scale feature matching technique from Assignment 2 to identify features in the following two images:



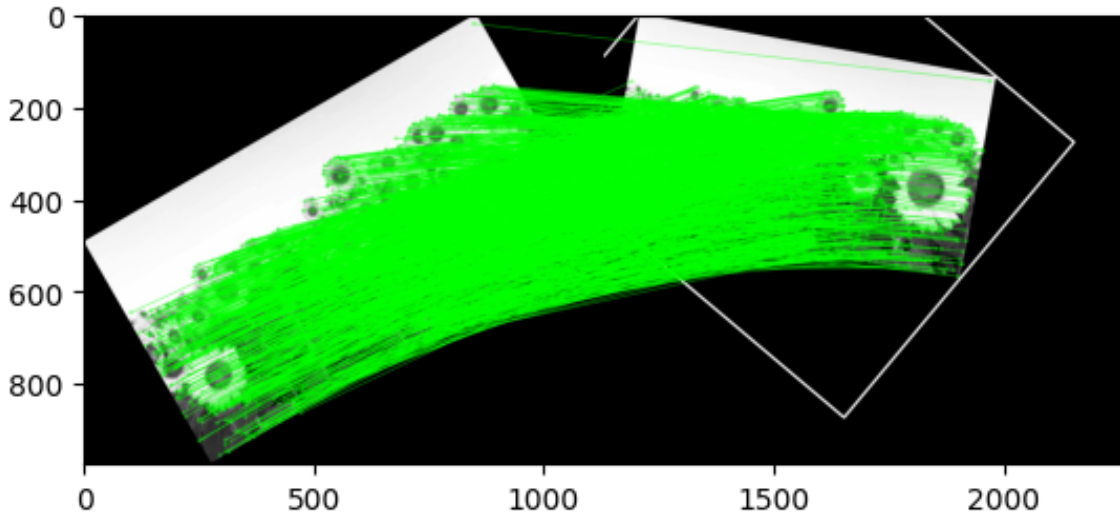
Using those features, I found related pairs and used those features to reconstruct the following homography matrix:

$$H = \begin{bmatrix} 0.613 & -0.524 & 168.85 \\ 0.518 & 0.617 & -153.21 \\ 1.72 \times 10^{-5} & -8.55 \times 10^{-6} & 1.0 \end{bmatrix}$$



4.4 - Feature Matching with OpenCV

Here is the result image I created by following the provided [OpenCV feature matching tutorial](#):



Question: How does the performance (e.g., in terms of the number of features or accuracy of matches) of the OpenCV system compare to the system you implemented? How much faster (roughly, I do not need a precise number) is the OpenCV system compared to yours?

The OpenCV system was much faster than my own method, fully processing hundreds of matches in 2.0 seconds on my machine (while my own method took 12 minutes to find significantly fewer matches). As you can see here, the accuracy of the reconstruction was also quite a bit better, with near-zero difference for a large majority of the image space.

